Palm OS Programming Recipes

Setting Receive Wakeup Handler using the New Serial Manager

With the New Serial Manager (Palm OS 3.5 and above), your application can respond to incoming serial port data by means of a callback function. When the serial port receives a certain amount of data, the Palm OS invokes your callback function. To enable this process, your application must do the following:

- Define a callback function that processes the incoming serial port data.
- Register that callback function with the Palm OS, indicating the number of bytes that the serial port should receive before the Palm OS invokes your callback function.
- Prime that callback function before each invocation by the Palm OS.

The New Serial Manager saves your application the trouble of polling the serial port for new data. In a sense, the serial port "wakes up" when data is received. At that point, the interrupt service routine (ISR) servicing the serial port invokes your callback function. For this reason, the callback function is referred to as a "receive wakeup handler." Please note that this kind of serial port wakeup is different from the sleeping and waking of the processor. Do not confuse the two kinds of wake up.

Receive Wakeup Handler: The Steps

To set up and define a New Serial Manager receive wakeup handler, your application code should include the following steps. Note that these instructions assume that you understand the basics of serial programming with the Palm OS.

1. Include the appropriate header files in your project:

   ```
   #include <SerialMgr.h>
   ```

2. Open the serial port, using `SrmOpen`. This function call can open either the serial or the IR port.

   For serial port use:

   ```
   error = SrmOpen(serPortCradlePort, baud, &portId);
   ```

   For IR port use:

   ```
   error = SrmOpen(serPortIRPort, baud, &portId);

   error = SrmControl(portId, srmCtlIrDAEnable, 0L, 0L);
   ```

3. If you are dealing with IR, disable or enable the IrDA receiver, as appropriate: if you will be sending data, disable the reciever; if you will be receiving data, enable the receiver. This step is necessary to prevent feedback.

   ```
   error = SrmControl(portId, srmCtlRxDisable, 0L, 0L);
   ```

or

```
error = SrmControl(portId, srmCtlRxEnable, 0L, 0L);
```

4. Register the callback function with the Palm OS. To do this, set the receive wakeup handler to the callback function, using `SrmSetWakeupHandler`.

   ```
   error = SrmSetWakeupHandler(portId,
   (WakeupHandlerProcPtr)RecvInthandler, (UInt32)refCon);
   ```

   Use the last parameter to pass data to the callback function. In the example, I am passing in the unique port ID so that the callback function can use it.

5. Before the Palm OS can invoke your receive wakeup handler, you must prime the handler, using `SrmPrimeWakeupHandler`.

   Every time a wakeup handler is called, it must be *reprimed* in order to be called again.

   ```
   error = SrmPrimeWakeupHandler(portId, bytes);
   ```

   Set `bytes` to the number of bytes that must be received before the receive wakeup handler is called. Typically, it is set to 1.

6. Code your callback function. It will look something like this:

```
static void RecvIntHandler(UInt32 refCon)
{
    Err err = 0;
    UInt32 count;
    //due to the limited stack space,
    //it's better to pass this as an incoming parameter
    UInt8* bufP;


    while (true) {
    //read the data
    err = SrmReceiveWindowOpen(portId, &bufP, &count);
    if (err)
        SrmClearErr(portId);
    if (!count)
        break;

    if (bufP)
    {
                //Process the data here.
    }

    //Let the New Serial Manager know that we're done with the data.
    SrmReceiveWindowClose(portId, count);
    }

    //Reprime the wakeup handler
```

```
    err = SrmPrimeWakeupHandler(portId, bytes);
}
```

NOTE: Because the receive wakeup handler is called during interrupt time, you should code it with special care. See Coding Restrictions in the Callback Function below.

7. Now that yoyou can cr callback function is coded, registered, and primed, continue coding your application where you left off in step 3 above. You can rest assured that at run time, the Palm OS will invoke your callback function when the serial port has received the number of bytes you specified.

8. Close the port and perform cleanup. The nature of your application will determine where you do these things. Remember, however, that an open serial port reduces battery life and that you should not leave a port open for long periods of time.

```
error = SrmClose(portId);
```

Notes Coding Restrictions in the Callback Function

The receive wakeup callback function is invoked by an interrupt service routine (ISR). When coding the callback function, keep the following in mind:

- The interrupt could happen at any time, even interrupting a function call your application was making.
- An ISR has very limited functionality available; thus, your callback function should not allocate or free memory nor make calls to the storage heap.
- Your callback function should not call Palm OS system functions that block the system in any way.
- Your callback function should not perform lengthy processing, because the ISR needs to return very quickly.

An excellent strategy to handle these restrictions is to have your callback function enqueue a custom event. You can then handle it during your application's normal event loop.

New Serial Manager vs. Serial Manager

The New Serial Manager is available in Palm OS 3.5 and above and replaces the older manager, which was known simply as the Serial Manager. Functions that begin with the prefix Srm, as in `SrmControl()`, belong to the New Serial Manager. Functions that begin with the prefix Ser, as in `SerControl()`, belong to the Serial Manager. Each manager has a separate chapter in the Palm OS SDK Programmer's Reference.

The portID parameter

The steps above contain several calls to New Serial Manager functions. The parameter `portID` appears in all of them. It is an unsigned integer that uniquely identifies a specific opened port. The uniqueness of this number is important because the New Serial Manager supports the possibility of multiple physical and virtual serial devices. The call to `SrmOpen` uses the `portID` parameter to return a unique identifier for that port. Then you pass in `portID` when calling other New Serial Manager functions.